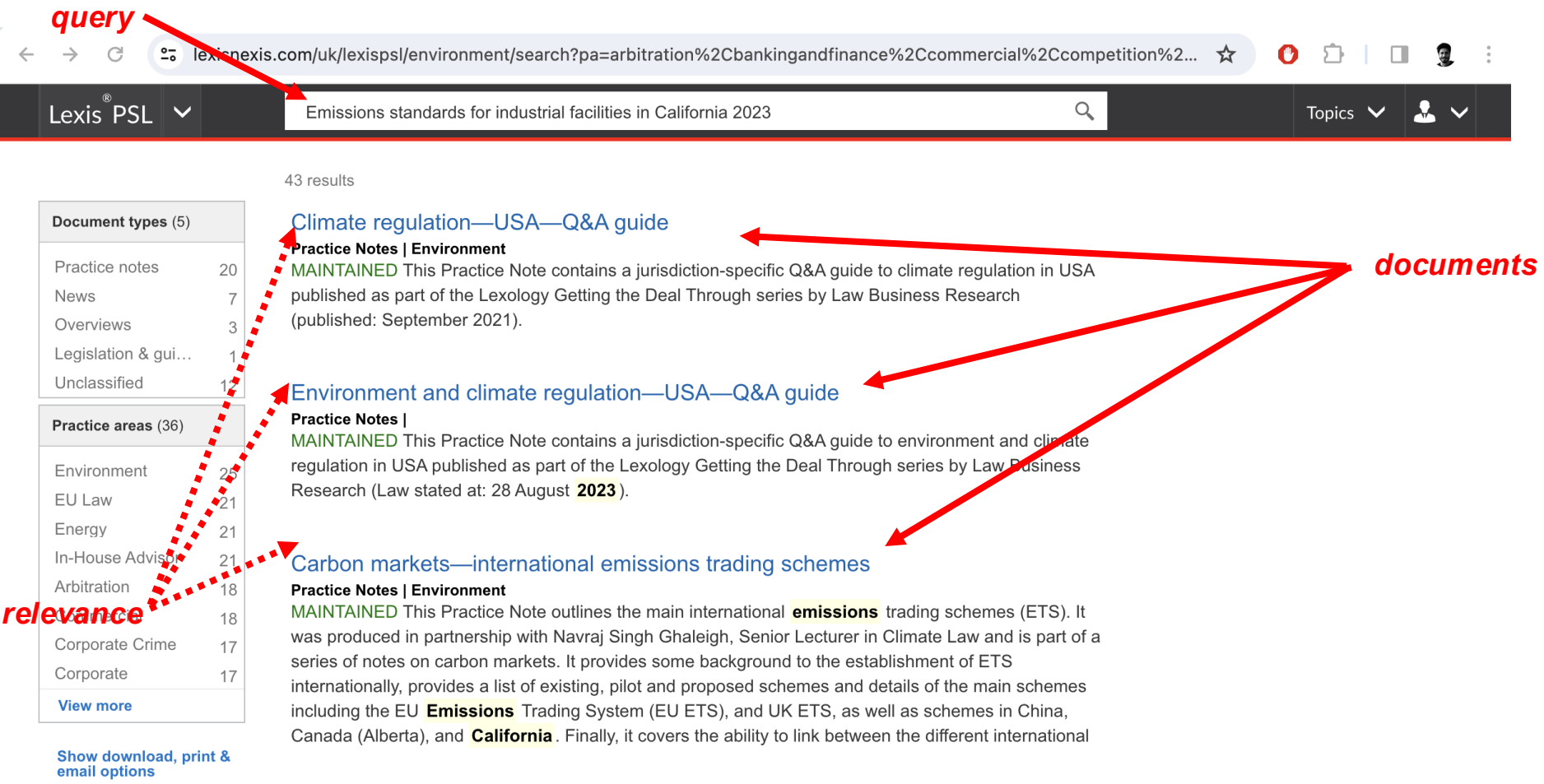


GESTIÓN DE SISTEMAS DE DATOS MASIVOS

Indexing and Information Retrieval

1. Storage vs Information Retrieval
2. Indexing and Text Representation
3. Retrieval and Ranking Models
4. Retrieval Evaluation
5. Use Case: Elasticsearch 8.7.0

Information Retrieval Example



Data Storage vs. Information Retrieval

Data storage systems	Information retrieval systems
Stora and manage data	Find relevant information
Exact or structured conditions	Textual matching and relevance
Results satisfy constraints	Results are ranked
Indexes optimize data access	Inverted indexes optimize text search
Correctness and consistency	Relevance, coverage and latency

From Data Queries to Information Needs

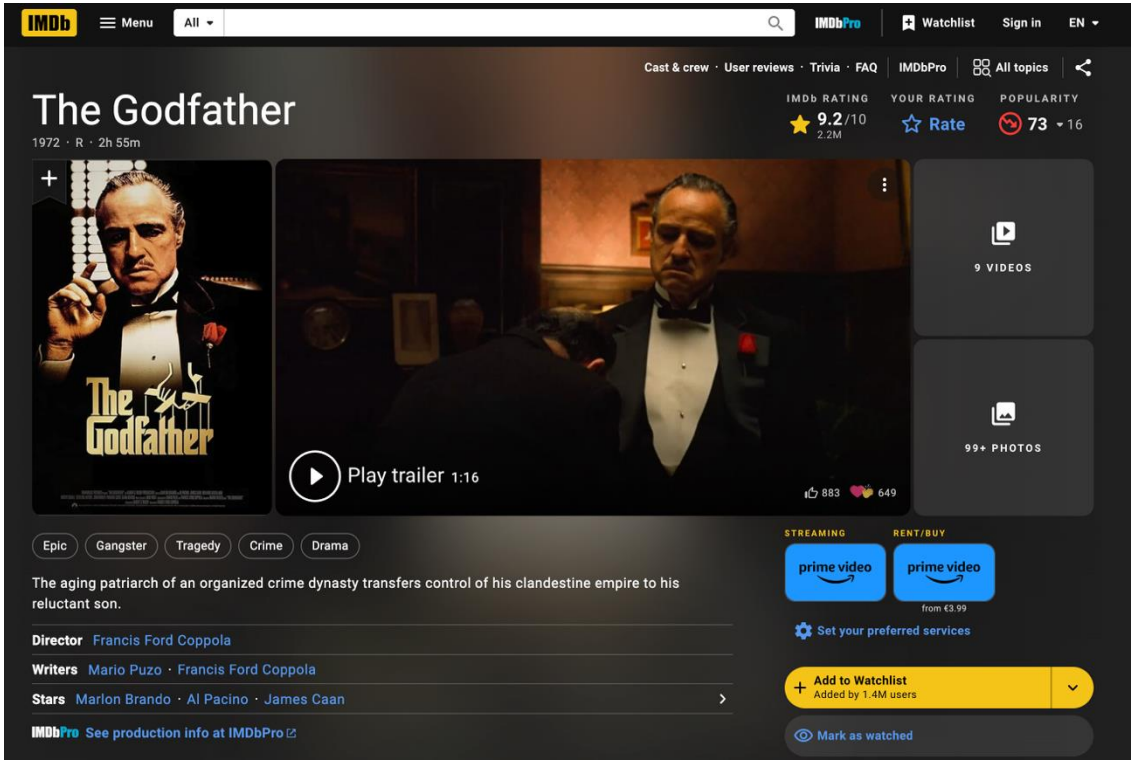
Information retrieval aims to **find documents** that help satisfy a **user's information need**.

- **Information need**: what the user actually wants to know.
- **Query**: an expression of that need.
- **Document collection (corpus)**: The set of documents available for search.
- **Relevance**: how useful each document is for satisfying the need.
- **Ranking**: ordering documents by estimated relevance.

Matching query terms does not necessarily mean satisfying the information need.

Exact Matching vs. Ranked Retrieval

- **Exact retrieval:** find reviews with rating = 5.
- **Full-text retrieval:** find reviews discussing exciting movies.
- **Hybrid retrieval:** find highly rated reviews discussing exciting movies.

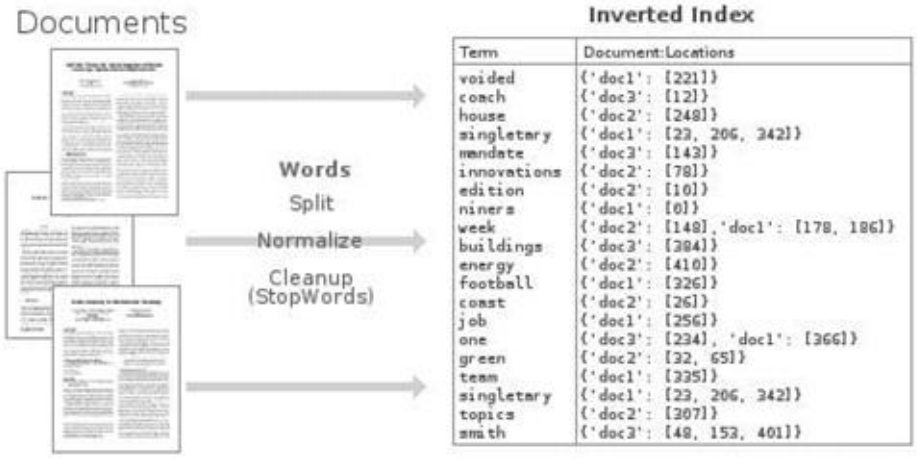


Indexing and Information Retrieval

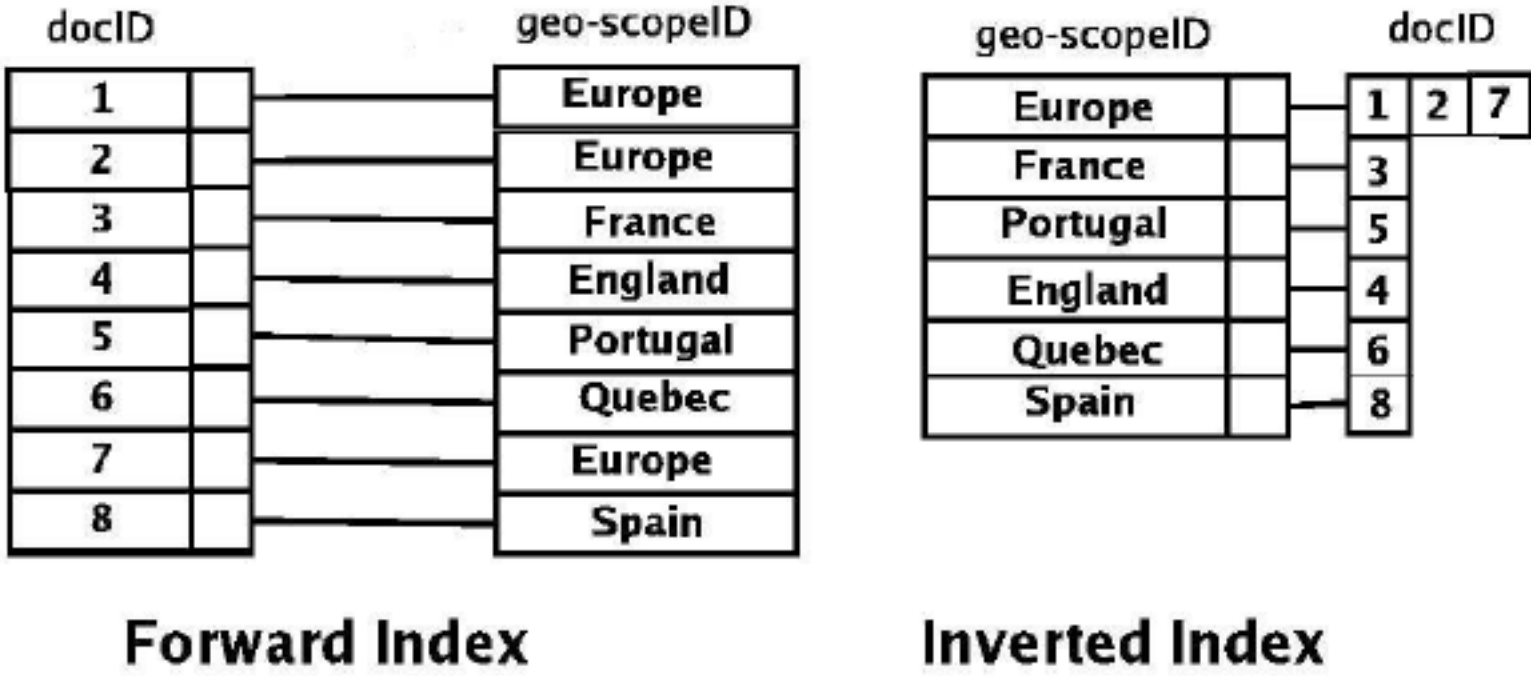
1. Storage vs Indormation Retrieval
2. Indexing and Text Representation
3. Retrieval and Ranking Models
4. Retrieval Evaluation
5. Use Case: Elasticsearch 8.7.0

The Inverted Index

- The content of each row (or document) is split up, and each individual entry (in this case each word) points back to any documents that it was found within.
- This inverted-index data structure allows us to very quickly find, for example, all of the documents where "*football*" was mentioned.



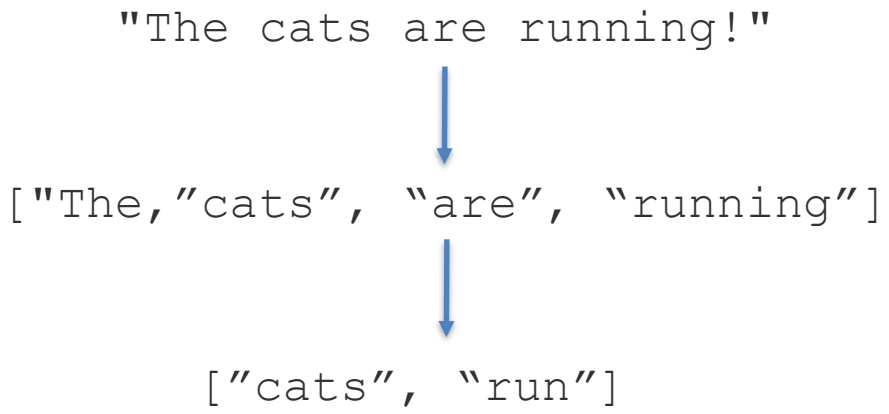
Inverted Index



Text Analysis: From Raw Text to Searchable Terms

Text analysis transforms raw text into a set of searchable terms that can be stored in an inverted index.

- **Tokenization**: splits text into individual units (tokens).
- **Normalization**: standardizes terms to reduce linguistic variations.
- **Filtering**: removes or transforms tokens according to retrieval needs.



Text Analysis Techniques

Different techniques can be combined to improve term matching and retrieval effectiveness.

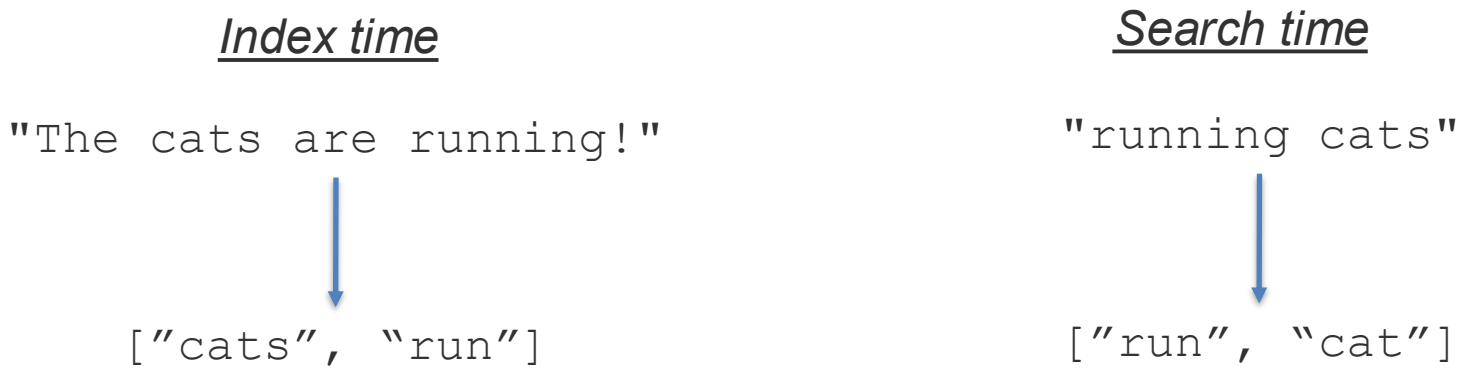
Technique	Purpose	Example
Lowercasing	Reduce case differences	Movie -> movie
Sopt-Word removal	Remove frequent, less informative words	the, is, a…
Stemming	Reduce words to common stems	running -> run
Lemmatization	Reduce words to dictionary forms	better -> good
Synonym expansión	Match alternative expressions	film -> movie

Key idea: Text analysis determines which terms can match during retrieval.

Index-Time vs. Search-Time Analysis

Text analysis can be applied at two different stages of the retrieval process.

- **Index time:** document text is analyzed before being indexed.
- **Search time:** queries are analyzed to match indexed terms.



Structured vs. Full-Text Representation

Not all document fields require the same indexing and retrieval strategies.

	Structured fields	Full-text fields
Purpose	Exact matching and filtering	Textual information retrieval
Representation	Original or normalized values	Analyzed terms
Retrieval	Equality, ranges, sorting	Term matching and ranking
Examples	Movie title, rating, date	Movie reviews, descriptions

Key idea: The information to be retrieved determines how data should be represented and indexed.

Indexing and Information Retrieval

1. Storage vs Indormation Retrieval
2. Indexing and Text Representation
3. **Retrieval and Ranking Models**
4. Retrieval Evaluation
5. Use Case: Elasticsearch 8.7.0

Boolean Retrieval vs. Ranked Retrieval

Information retrieval models determine how documents are selected and ordered in response to a query.

- **Boolean Retrieval**: selects documents satisfying logical conditions (AND, OR, NOT).
- **Ranked Retrieval**: assigns relevance scores to matching documents and orders them accordingly.

Example:

D1 amazing science fiction movie

D2 boring science fiction movie

D3 amazing historical drama

Query "amazing science fiction"

Boolean AND → D1
Boolean OR → D1, D2, D3
Ranked retrieval → D1 prioritized by estimated relevance

Term Weighting: TF and IDF

Not all terms contribute equally to document relevance.

- **Term Frequency (TF):** how frequently a term appears in a document.
- **Document Frequency (DF):** how many documents contain a given term.
- **Inverse Document Frequency (IDF):** gives greater importance to terms appearing in fewer documents.

TF-IDF combines term frequency and inverse document frequency to estimate term importance:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{DF(t)}$$

Vector Space Model

Documents and queries are represented as vectors in a shared term space.

- **Vector representation:** each dimension corresponds to a vocabulary term.
- **Term weights:** TF-IDF can represent the importance of each term.
- **Cosine similarity:** measures the similarity between document and query vectors.

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

Key idea: Documents can be ranked by their similarity to the query in a weighted term space.

BM25: Improving Lexical Ranking

BM25 estimates document relevance using term statistics and document length normalization.

- **Term Frequency Saturation:** repeated occurrences increase relevance with diminishing returns.
- **Inverse Document Frequency:** rare terms contribute more to the score.
- **Document Length Normalization:** adjusts scores to account for differences in document length.

$$\mathrm{BM25}(d,q) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{avgdl}\right)}$$

Key idea: BM25 ranks documents by estimated lexical relevance, rather than simply counting matching terms.

Combining Ranking and Filtering

Search systems can combine exact constraints with relevance-based retrieval.

- **Filtering:** determines whether a document satisfies mandatory conditions.
- **Ranking:** estimates how well a matching document satisfies the information need.
- **Combined retrieval:** ranks relevant documents while enforcing structured constraints.

Key idea: Filtering restricts the candidate documents; ranking determines their relative relevance.

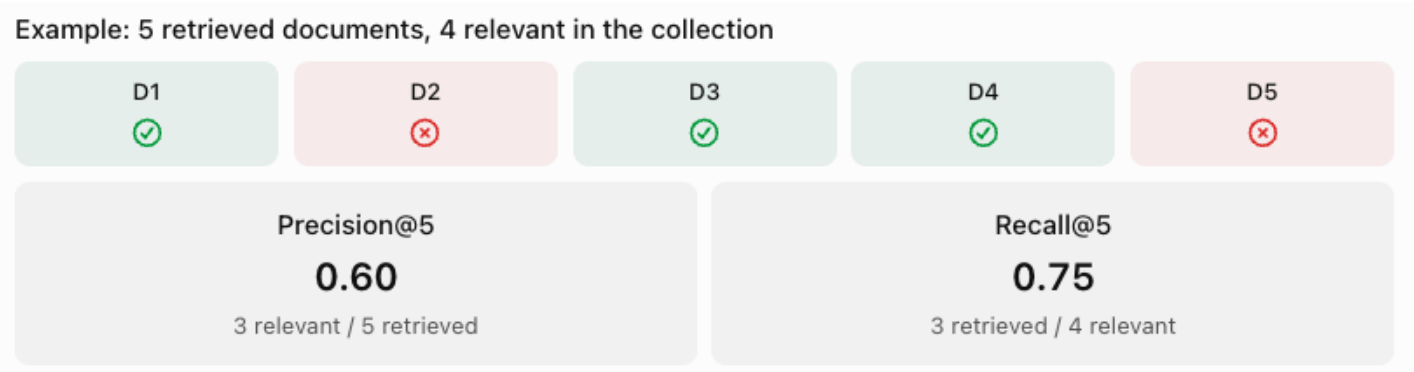
Indexing and Information Retrieval

1. Storage vs Indormation Retrieval
2. Indexing and Text Representation
3. Retrieval and Ranking Models
4. **Retrieval Evaluation**
5. Use Case: Elasticsearch 8.7.0

Evaluating Retrieval Quality: Precision and Recall

Retrieval effectiveness measures how well search results satisfy users' information needs.

- **Ground truth:** a set of queries with human-assessed relevant documents.
- **Precision@k:** proportion of relevant documents among the top-k results.
- **Recall@k:** proportion of all relevant documents retrieved in the top-k results.

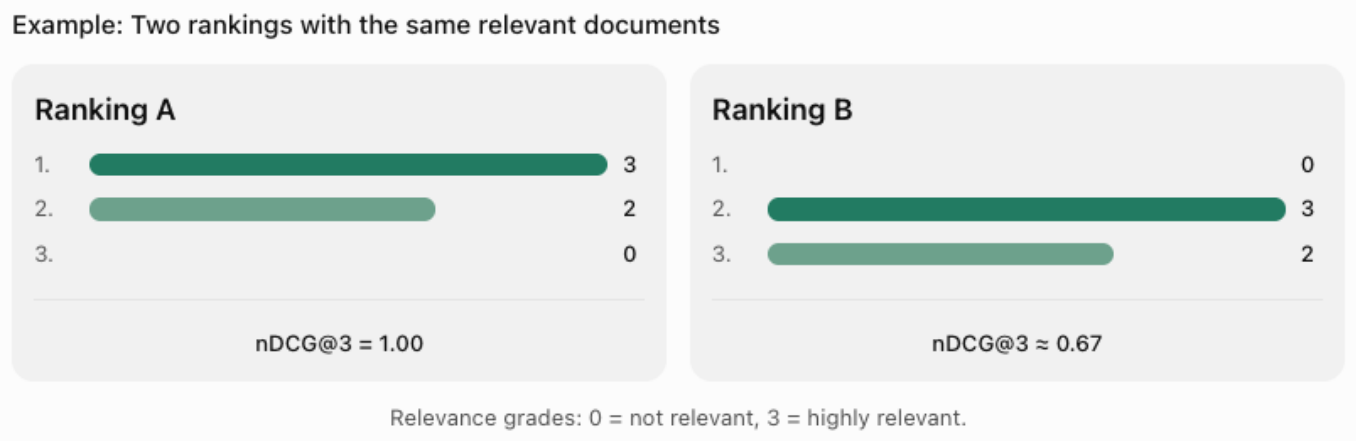


Key idea: Precision measures result accuracy, while recall measures coverage of relevant information.

Evaluating Ranking Quality: nDCG

Precision and recall measure which relevant documents are retrieved, but not how effectively they are ranked.

- **Graded relevance:** documents can have different levels of relevance (e.g., 0–3).
- **Discounted Cumulative Gain (DCG):** rewards highly relevant documents appearing earlier.
- **Normalized DCG (nDCG):** compares the ranking against an ideal ordering.



Key idea: Ranking quality depends not only on retrieving relevant documents, but also on placing the most relevant ones first.

Indexing and Information Retrieval

1. Storage vs Indormation Retrieval
2. Indexing and Text Representation
3. Retrieval and Ranking Models
4. Retrieval Evaluation
5. **Use Case: Elasticsearch 8.7.0**

Databases vs Search Engines

- **Databases** (e.g., SQL databases or NonSQL Stores) store structured or semi-structured data with fixed or flexible schemas.
- **Search engines** (e.g., Elasticsearch) go beyond storage by providing powerful ***additional search-oriented*** capabilities.
- Databases focus on **storing and querying application data**.
  - Boolean matching
- Search engines focus on **retrieving and ranking relevant information**.
  - Their capabilities may overlap.
  - Based on a relevance score.
  - Textual Search engines analyze text during indexing, allowing for advanced **tokenization** and **text analysis**.
- Databases are ideal for **massive data storage**, while search engines excel at retrieving and analyzing **large volumes of information**.

Document-oriented Databases

- Data is stored as **documents** instead of rows.
  - JavaScript Object Notation (JSON), a lightweight data format that is easy to read and write.
- Each document is **self-contained** and can store complex, **nested structures**.
- **Key Characteristics**:
  - Schema-less structure
  - Nested and hierarchical data support
  - Data represented as documents
    - self-contained units
- **Advantages**:
  - Schema flexibility
  - Fast reads and writes for hierarchical data
- **Drawbacks:**
  - Querying specific fields may require retrieving the whole document.
  - Potential for Data Redundancy
  - Embedding data (e.g., related documents) can lead to duplication of information.
  - Lack of Strong Consistency
  - In distributed systems, NoSQL databases (including document stores) may prioritize availability over consistency (CAP theorem)
  - Complexity in Transactions
  - Limited support for multi-document transactions compared to SQL.

```
{
  "_id": "5cf0029caff5056591b0ce7d",
  "firstname": "Jane",
  "lastname": "Wu",
  "address": {
    "street": "1 Circle Rd",
    "city": "Los Angeles",
    "state": "CA",
    "zip": "90404"
  }
  "hobbies": ["surfing", "coding"]
}
```

Source: MongoDB

Elasticsearch: a Search Engine and Document Store

- Elasticsearch is a **distributed**, **open-source search** and **analytics engine** built on top of Apache Lucene, offering both **full-text search and document storage**.

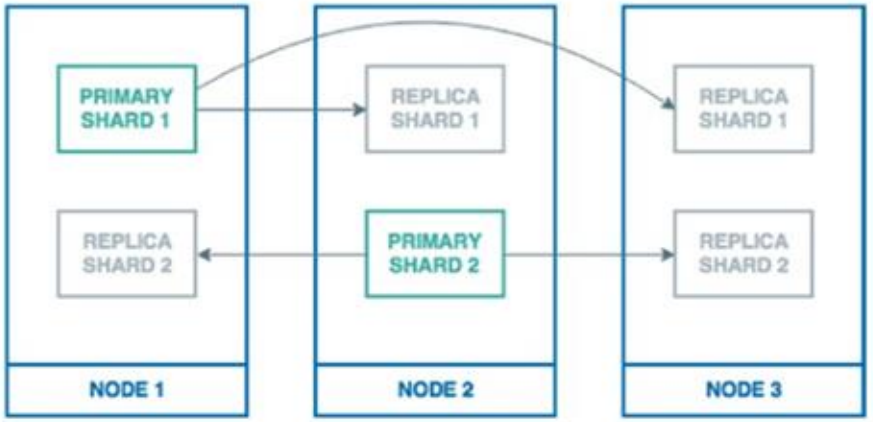
- **Core Capabilities**
  - Full-text search with **relevance scoring** and **tokenization**.
  - **Document-oriented storage** using JSON documents.
  - **Distributed** and **scalable** architecture for handling large datasets.
  - **Near real-time** indexing, search, and updates.

- **Primary Use Cases**
  - Search across large datasets (e.g., logs, documents, websites)
  - **Data storage** and **retrieval** for structured and semi-structured documents.
  - Analytics on structured and unstructured data



Key Concepts in Elasticsearch

- **Index**:
  - Comparable to a **database** in SQL.
  - An index stores **documents** and has a specific **structure**.
  - Each index has **mappings**, which define the structure of the documents (field types, text analysis).
- **Document**:
  - A single **unit of data** stored in JSON format.
  - Each document is a collection of **fields**.
- **Shards**:
  - An index is divided into shards, which allows Elasticsearch to store and search large datasets by **distributing** data across multiple nodes
  - Sharding allows for **horizontal scaling**.
  - A shard is a self-contained search engine and is a Lucene index.
- **Replicas**:
  - Copies of shards that provide **fault tolerance** and **load balancing**.
  - Replicas ensure **high availability** in case of node failure.



Source: Elasticsearch

Installing Elasticsearch *(by Docker)*

- Clone the GitHub Repository:

```
git clone https://github.com/gestbd/Elasticsearch.git
cd Elasticsearch
```

- Create an external network

```
docker network create gestbd_net
```

- Run Docker Compose to Set Up Elasticsearch:

```
docker compose up
```

- Access Elasticsearch: the service will be available at:
  - Elasticsearch: http://localhost:9200

Elasticsearch: docker-compose.yml

```
version: '3.1'
services:
    elasticsearch:
      image: elasticsearch:8.7.0
      ports:
        - 9200:9200
      environment:
        - discovery.type=single-node
        - ES_JAVA_OPTS=-Xms1g -Xmx1g
        - xpack.security.enabled=false
      volumes:
        - ./es-data:/usr/share/elasticsearch/data
      networks:
        - gestbd_net

networks:
    gestbd_net:
      external: true
```

Installing Jupyter Notebook Using Docker

- Clone the GitHub Repository:

```
git clone https://github.com/gestbd/Jupyter.git
cd Jupyter
```

- Run Docker Compose to Set Up Jupyter Notebook:

```
docker compose up
```

- Access Jupyter Notebook:
  The service will be available at:
  - Elasticsearch: http://localhost:8888

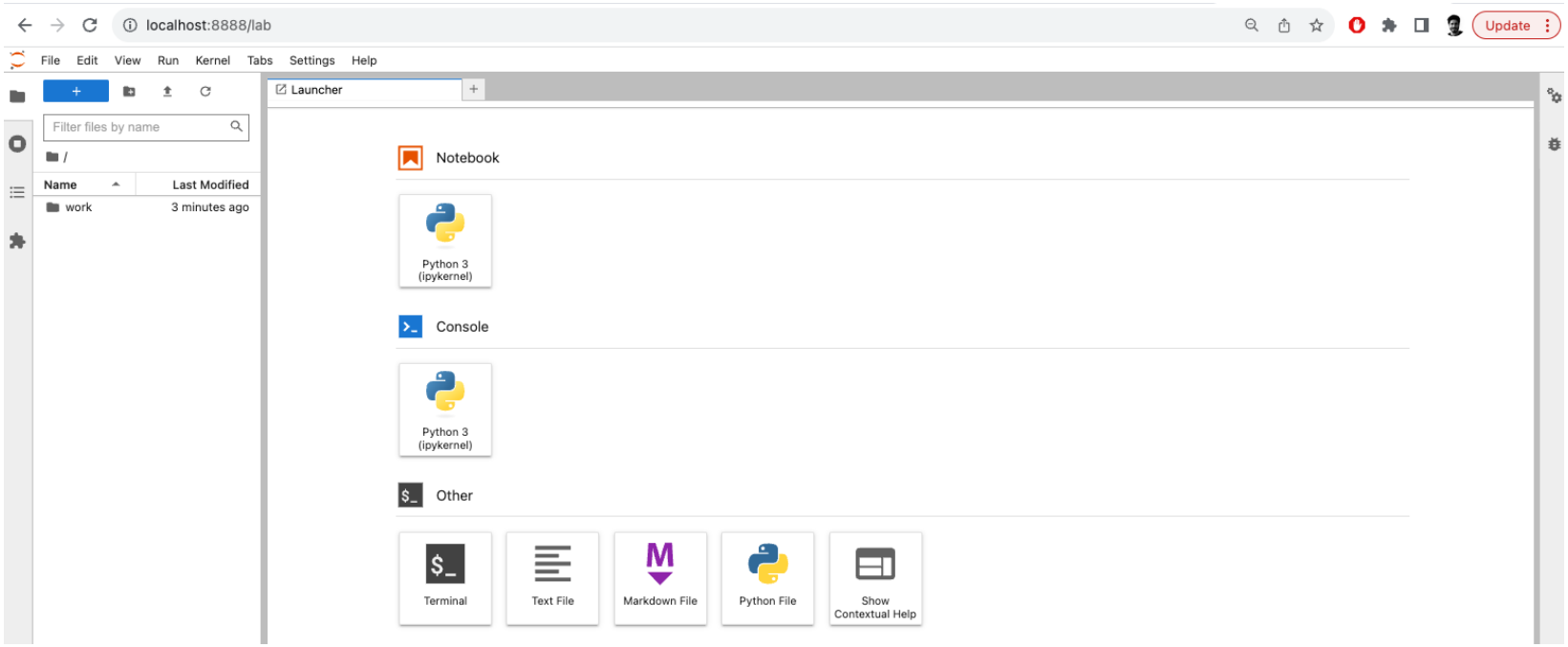
Jupyter docker-compose.yml

```
services:
    jupyter:
          image: jupyter/scipy-notebook:python-3.8
        ports:
          - 8888:8888
        volumes:
          - "./notebooks:/home/jovyan/work/notebooks"
        command: ["start-notebook.sh","--NotebookApp.token=''","--
NotebookApp.password=''"]
        networks:
          - gestbd_net

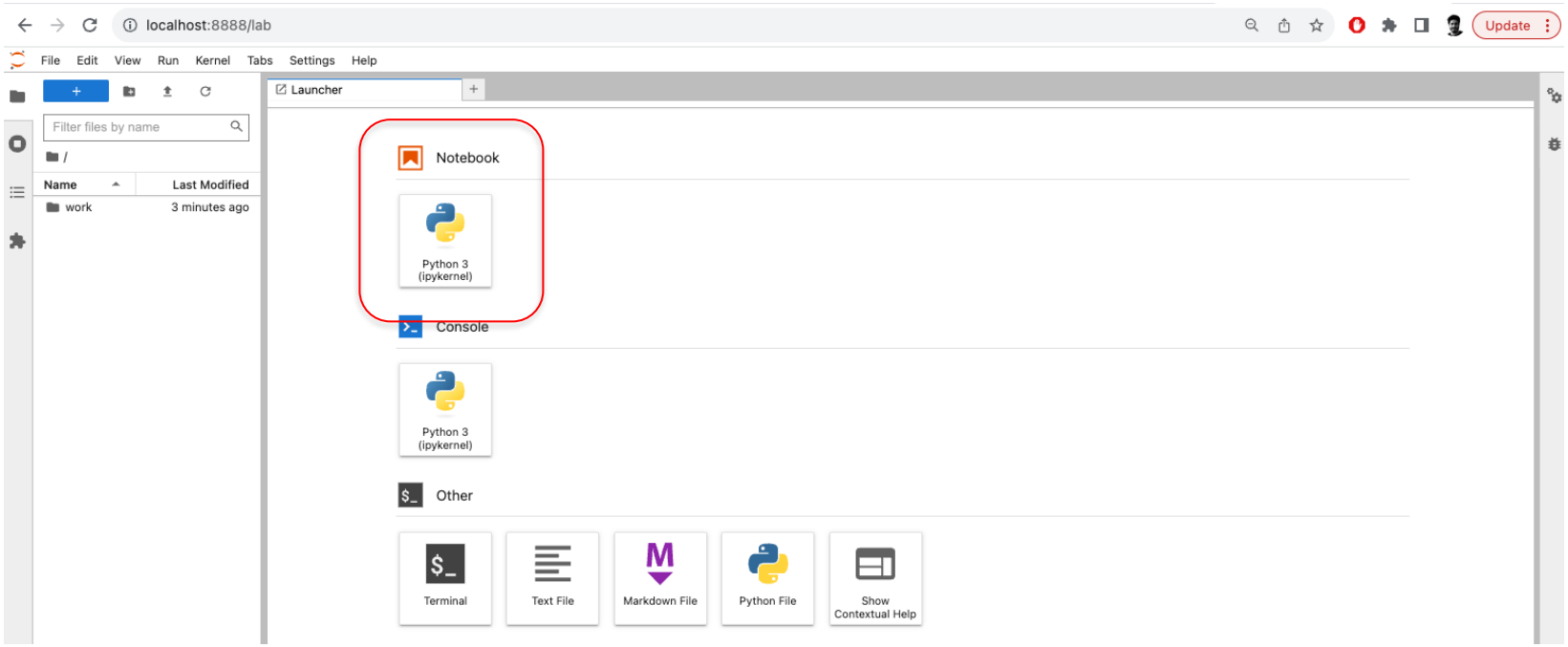
networks:
    gestbd_net:
      external: true
```

Jupyter Lab

- The service is available at http://localhost:8888



Create a new Notebook



Install the Python Client

- To install the 8.7.0 version of the client, run the following command:

```
!pip install "elasticsearch==8.7.0"
```

```
[2]: !pip install "elasticsearch==8.7.0"
Collecting elasticsearch==8.7.0
  Downloading elasticsearch-8.7.0-py3-none-any.whl (387 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 387.9/387.9 kB 11.6 MB/s eta 0:00:00
Collecting elastic-transport<9,>=8
  Downloading elastic_transport-8.17.1-py3-none-any.whl (64 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.0/65.0 kB 27.5 MB/s eta 0:00:00
Requirement already satisfied: certifi in /opt/conda/lib/python3.8/site-packages (from elastic-transport<9,>=8->elastic
search==8.7.0) (2022.9.24)
Requirement already satisfied: urllib3<3,>=1.26.2 in /opt/conda/lib/python3.8/site-packages (from elastic-transport<9,>
=8->elasticsearch==8.7.0) (1.26.11)
Installing collected packages: elastic-transport, elasticsearch
Successfully installed elastic-transport-8.17.1 elasticsearch-8.7.0
```

Connect to the Server

- Connect to the Elastic server using the Elasticsearch endpoint.

```
from elasticsearch import Elasticsearch

es = Elasticsearch("http://elasticsearch:9200")

# Verifica la conexión
print(es.info().body)
```

[3]:
```
from elasticsearch import Elasticsearch

es = Elasticsearch("http://elasticsearch:9200")

# Verifica la conexión
print(es.info().body)
```

```
{'name': 'e3392c40e133', 'cluster_name': 'docker-cluster', 'cluster_uuid': 'mNVwAhZLQOq7mkNAF5qXbA', 'version': {'numbe
r': '8.7.0', 'build_flavor': 'default', 'build_type': 'docker', 'build_hash': '09520b59b6bc1057340b55750186466ea715e30
e', 'build_date': '2023-03-27T16:31:09.816451435Z', 'build_snapshot': False, 'lucene_version': '9.5.0', 'minimum_wire_c
ompatibility_version': '7.17.0', 'minimum_index_compatibility_version': '7.0.0'}, 'tagline': 'You Know, for Search'}
```

Document Indexing and Text Analysis

Data Preprocessing

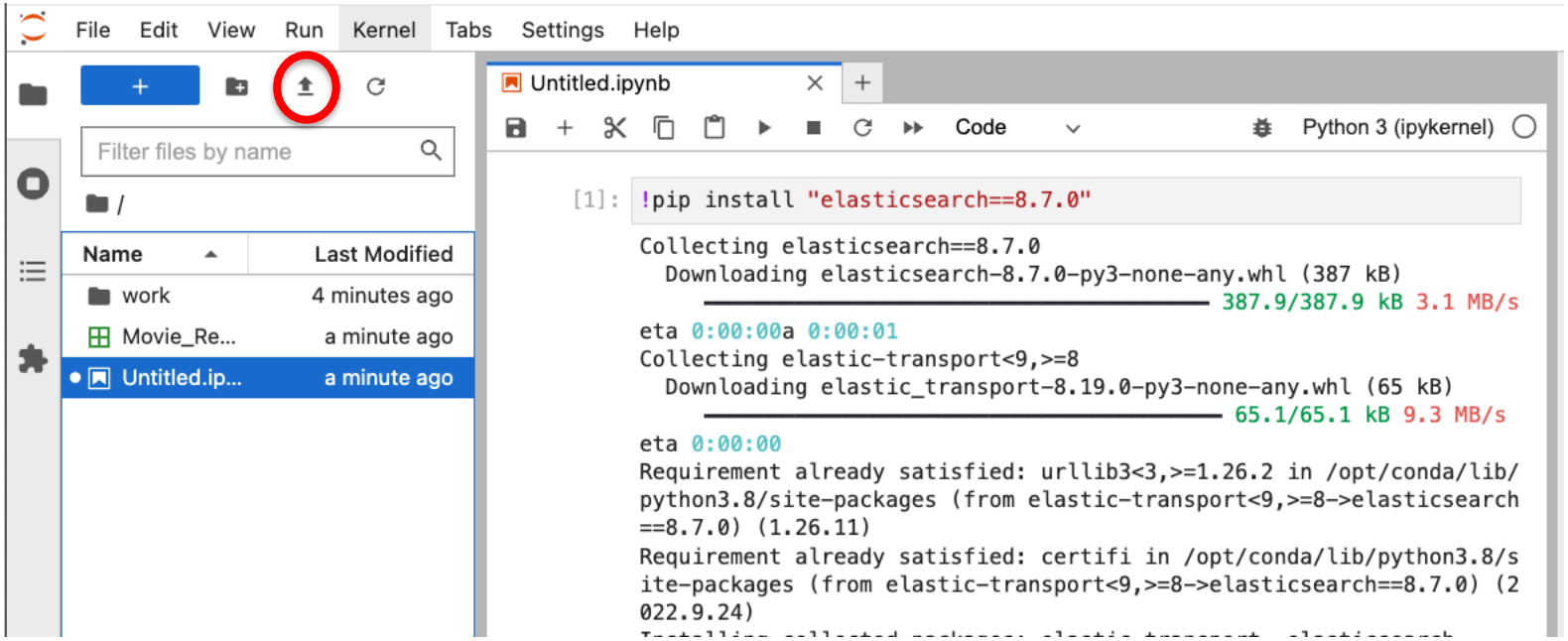
When working with textual data like movie reviews, it's important to preprocess the data before storing it in Elasticsearch to ensure consistency and optimize search performance

Tools Required:

- Python

Load CSV file

- Download `Movie_Reviews.csv` from Moodle
- Upload to the Jupyter notebook



Handling Missing Values

- Missing values in fields like movie_title or review_text can lead to indexing errors or incomplete documents.
- Check for missing values and handle them appropriately (e.g., replacing with a default value, or excluding the document).

```
import pandas as pd

# Load CSV file
df = pd.read_csv('Movie_Reviews.csv')

# Check for missing values
df.fillna({'movie_title': 'Unknown', 'review_text': 'No review provided'},
inplace=True)
df.shape
```

Text Normalization

- Text data in reviews can have inconsistent capitalization, special characters, or whitespace.
  - Normalizing text ensures that searches are more effective.
- Convert all text to lowercase, remove extra whitespace, and strip unwanted characters (like punctuation).

```
import re

def clean_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation
    text = re.sub(r'[^\w\s]', '', text)
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

df['review_text'] = df['review_text'].apply(clean_text)
```

Removing Stop Words

- Stop words (e.g., "and", "the", "is") are common words that often don't add much value in search queries.
- You can remove them to reduce noise in the dataset, but be cautious as this can affect certain searches.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

df['review_text'] = df['review_text'].apply(lambda x: ' '.join([word for word
in x.split() if word not in stop_words]))
```

- Ensure you have nltk installed and stopwords downloaded:

```
pip install nltk
python -m nltk.downloader stopwords
```

Handling Ratings (Numeric Data)

- Ensure that the rating field is a numeric value for easier range queries.
- Convert non-numeric entries (if any) to NaN or a default value, then cast the column to an integer or float.

```
df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
df['rating'].fillna(0, inplace=True)  # Replace missing ratings with 0
```

Handling Special Characters

- Special characters (e.g., emojis, accented characters) might not be searchable in Elasticsearch without proper handling.
- Remove or normalize these characters during preprocessing

```
!pip install Unidecode
```

```
import unidecode

df['review_text'] = df['review_text'].apply(lambda x: unidecode.unidecode(x))
```

Creating Unique Identifiers

- Elasticsearch requires a unique identifier for each document.
  - If your dataset doesn't have one, you can generate unique IDs based on the row index or a combination of columns.
- Use the row index as the **review_id**, or generate a hash based on multiple columns.

```
df['review_id'] = df.index + 1  # Create a simple numeric ID
```

Handling Duplicates

- Duplicated reviews can skew search results and analytics.
- Check for duplicates based on fields like **review_text** or **movie_title**.

```
df.drop_duplicates(subset=['review_text'], inplace=True)
```

Converting to JSON Format

- Elasticsearch requires documents to be sent as JSON.
- Convert the DataFrame rows to a dictionary format, then to JSON for easy indexing.

```
records = df.to_dict(orient='records')  # DataFrame to list of dictionaries
```

Data Indexing

What is Indexing in Elasticsearch?

- Indexing is the process of **storing documents** in an Elasticsearch index, making them available for search.

- Core Concept:
  - **Index**: A collection of documents.
  - **Document**: A JSON object containing data.
  - **Field**: Each document is composed of fields (key-value pairs).

- Analogy:
  - Similar to **inserting rows** into a table in a relational database, but with flexible schema.

Inspecting Tokens with the Analyze API

Elasticsearch provides the **Analyze API** to inspect how text is transformed into **searchable tokens**.

- Compare different analyzers using the same input text.
- Observe the effects of tokenization, stop-word removal and stemming.
- Understand how analyzer configuration affects the terms available for searching.

Example: Comparing Standard and English Analyzers

```
text = "The cats are running!"

for analyzer in ["standard", "english"]:
    response = es.indices.analyze(
        analyzer=analyzer,
        text=text
    )

    tokens = [t["token"] for t in response["tokens"]]
    print(f"{analyzer}: {tokens}")
```

Expected output:

```
standard: ['the', 'cats', 'are', 'running']
english:  ['cat', 'run']
```

Load the Data into Elasticsearch

- Load the movie reviews dataset from the provided CSV file
  - **Movie_Reviews.csv**.
- Create an index named **movie_reviews** in Elasticsearch.
- Insert each review as a document in the **movie_reviews** index, ensuring that each document contains the following fields:
  - **review_id** (unique identifier for each review)
  - **movie_title** (title of the movie)
  - **review_text** (the full review content)
  - **rating** (numeric score for the movie)
  - **reviewer_name** (name of the reviewer)

Load Data

```
import pandas as pd

# Load CSV file
df = pd.read_csv('Movie_Reviews.csv')

# Check for missing values
df.fillna({'movie_title': 'Unknown', 'review_text': 'No review provided'}, inplace=True)
df.shape
```

Prepare Data

```
df['rating'] = pd.to_numeric(df['rating'], errors='coerce')
df['rating'].fillna(0, inplace=True) # Replace missing ratings with 0
df['review_id'] = pd.to_numeric(df['review_id'], errors='coerce')
df.head()
```

	review_id	movie_title	review_text	rating	reviewer_name
0	1	The Pulp Fiction	It was decent. I recommend watching it if you ...	3	Clara
1	2	Avatar	An exciting and thrilling experience without a...	2	John
2	3	The Godfather	I found the movie quite underwhelming given th...	4	John
3	4	Pulp Fiction	This film is overrated. The plot is predictabl...	4	Alice
4	5	The Matrix	Too much focus on action; the characters were ...	2	Alice
...	...	...	...	...	...

Create an index

```
mapping = {
    "mappings": {
        "properties": {
            "review_id": { "type": "integer" },
            "movie_title": { "type": "keyword" },
            "review_text": { "type": "text" },
            "rating": { "type": "integer" },
            "reviewer_name": { "type": "keyword" }
        }
    }
}

index_name = 'movie_reviews'
if es.indices.exists(index=index_name):
    es.indices.delete(index=index_name)

es.indices.create(index=index_name, mappings=mapping["mappings"])
print(f"Index '{index_name}' created with mapping.")
```

Index Documents

```
import json
from elasticsearch.helpers import bulk

def bulk_index_data(es, data, index_name):
    batch_size = 50  # Reducir el tamaño del lote a 50
    for i in range(0, len(data), batch_size):
        batch = data[i:i+batch_size]
        actions = []
        for doc in batch:
            actions.append({
                "_index": index_name,
                "_id": doc['review_id'],
                "_source": doc
            })
        # Capturar la respuesta para verificar errores
        resp = bulk(es, actions, raise_on_error=True)
        print("Indexed:", resp[0], "Errors:", resp[1])

reviews = df.to_dict(orient='records')
bulk_index_data(es, reviews, index_name)
```

Refresh Index

```
es.indices.refresh(index="movie_reviews")
```

Querying and Exploiting Search Results

Search Data: Elasticsearch Query DSL

- Query DSL (Domain Specific Language) is a JSON-style query language for Elasticsearch that enables complex searching, filtering and aggregations.
- Core Capabilities:
  - Full-text search (analyzed fields)
  - Exact keyword search
  - Filtering (include/exclude documents)
  - Aggregations (metrics, buckets, pipelines)
- Structure:
  - Leaf query clauses (e.g., match, term, range) operate on individual fields.
  - Compound query clauses (e.g., bool, constant_score) combine multiple leaf or compound queries logically
- Two main contexts:
  - Query context → affects scoring / relevance.
  - Filter context → yes/no match, no scoring, often cached.
- Learn more and explore full details here:
  https://www.elastic.co/docs/explore-analyze/query-filter/languages/querydsl

Ranking Search Results with BM25

Elasticsearch uses **BM25** as its default **relevance scoring algorithm** for full-text search.

BM25 estimates how **relevant** a document is to a query based on three main factors:

- **Term Frequency (TF)**: How often the query terms appear in a document. More occurrences generally increase relevance, with diminishing returns.
- **Inverse Document Frequency (IDF)**: Rare terms across the document collection are more informative than common ones.
- **Document Length:** Shorter fields may receive higher scores when they contain the same matching terms.

Relevance Score (`_score`):

- Matching documents are ranked by their estimated relevance.
- Higher scores generally indicate better matches for the given query.
- Scores are query-dependent and do not represent probabilities.

Important: Query clauses contribute to relevance scoring, while filter clauses restrict results without affecting `_score`.

Inspecting Relevance Scores in Elasticsearch

Elasticsearch assigns a relevance score (`_score`) to each matching document in a full-text search.

- Results are ranked by `_score` in descending order by default.
- BM25 considers term frequency, term rarity and field length.
- The score is relative to the query and the indexed collection, not a probability of relevance.

Example: Search for movie reviews containing "amazing"

```
res = es.search(
    index="movie_reviews",
    size=5,
    query={
        "match": {"review_text": "amazing"}
    }
)

for rank, hit in enumerate(res["hits"]["hits"], start=1):
    doc = hit["_source"]
    print(f"{rank}. {doc['movie_title']} | Score: {hit['_score']:.3f}")
    print(f"   {doc['review_text'][:100]}...")
```

Q1: Exact Search

Exact Search with a Term Query

- Retrieve reviews whose `movie_title` exactly matches "Inception".
- Use a `term` query, since `movie_title` is mapped as a keyword field.
- Return up to 3 matching documents.

Remember: A term query does not analyze the search text.

Q1: Exact Search

Exact Search with a Term Query

- Retrieve reviews whose movie_title exactly matches "Inception".
- Use a term query, since movie_title is mapped as a keyword field.
- Return up to 3 matching documents.

Remember: A term query does not analyze the search text.

```
import json
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "term": {
            "movie_title": "Inception"
        }
    }
)
print(json.dumps(res.body, indent=4))
```

Q2: Full-Text Search

Full-Text Search with a Match Query

- Search for reviews matching "amazing" in the `review_text` field.
- Use a `match` query, since review_text is mapped as a text field.
- Elasticsearch analyzes the query before searching.
- Results are ranked by relevance (`_score`).

Q2: Full-Text Search

Full-Text Search with a Match Query

- Search for reviews matching "amazing" in the review_text field.
- Use a match query, since review_text is mapped as a text field.
- Elasticsearch analyzes the query before searching.
- Results are ranked by relevance (`_score`).

```
import json

res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "match": {
            "review_text": "amazing"
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q3: Numeric Filtering

Retrieve reviews with a rating of exactly 5.

Filtering with a **Term Query**

- The rating field is mapped as an integer.
- A term query can match an exact numeric value.
- The bool.filter clause restricts which documents are returned.
- Filters do not contribute to relevance scoring (_score).

Key idea: Filtering determines which documents satisfy a condition, without ranking them by relevance

Q3: Numeric Filtering

Retrieve reviews with a rating of exactly 5.

Filtering with a **Term Query**

- The rating field is mapped as an integer.
- A term query can match an exact numeric value.
- The bool.filter clause restricts which documents are returned.
- Filters do not contribute to relevance scoring (_score).

Key idea: Filtering determines which documents satisfy a condition, without ranking them by relevance

```
import json

res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "filter": [
                {"term": {"rating": 5}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q4: Combining Search and Filtering

Find reviews for the movie "`Titanic`" that mention "`boring`".

Combining **Query** and **Filter Contexts**

- Full-Text Search (must): Search for "boring" in the review_text field using a match query.
- Exact Filtering (filter): Restrict the results to reviews where movie_title is "Titanic" using a term query.
- Relevance Ranking: Only the full-text query contributes to _score; the filter does not affect relevance scoring.

Key idea: Combine relevance-based retrieval with exact constraints using a bool query.

Q4: Combining Search and Filtering

Find reviews for the movie "`Titanic`" that mention "`boring`".

Combining **Query** and **Filter Contexts**

- Full-Text Search (must): Search for "boring" in the review_text field using a match query.
- Exact Filtering (filter): Restrict the results to reviews where movie_title is "Titanic" using a term query.
- Relevance Ranking: Only the full-text query contributes to _score; the filter does not affect relevance scoring.

Key idea: Combine relevance-based retrieval with exact constraints using a bool query.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "must": [
                {"match": {"review_text": "boring"}}
            ],
            "filter": [
                {"term": {"movie_title": "Titanic"}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q5: Range Filtering

Retrieve reviews with a rating between 4 and 5 (inclusive).

Filtering with a **Range Query**

- Use a **range query** to select documents whose numeric values fall within a specified interval.
- `gte` **(greater than or equal)**: Defines the inclusive lower bound.
- `lte` **(less than or equal)**: Defines the inclusive upper bound.
- Place the range query inside **bool.filter** to restrict results without affecting relevance scoring.

Key idea: Range filters select documents based on numeric or date constraints, without ranking them by relevance.

Q5: Range Filtering

Retrieve reviews with a rating between 4 and 5 (inclusive).

Filtering with a **Range Query**

- Use a **range query** to select documents whose numeric values fall within a specified interval.
- **`gte` (greater than or equal)**: Defines the inclusive lower bound.
- **`lte` (less than or equal)**: Defines the inclusive upper bound.
- Place the range query inside **bool.filter** to restrict results without affecting relevance scoring.

Key idea: Range filters select documents based on numeric or date constraints, without ranking them by relevance.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "filter": [
                {
                    "range": {
                        "rating": {
                            "gte": 4,
                            "lte": 5
                        }
                    }
                }
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q6: Combining Exact and Full-Text Search

Find reviews written by "`John`" that mention "`excellent`".

Combining **Exact Matching** and **Full-Text Search**

- **Exact Filtering (`filter`)**: Select reviews where reviewer_name is exactly "John" using a term query.
- **Full-Text Search (`must`)**: Search for "excellent" in review_text using a match query.
- **Relevance Ranking:** Results are ranked according to their textual relevance, independently of the reviewer filter.

Key idea: Combine structured metadata constraints with full-text search in a single query.

Q6: Combining Exact and Full-Text Search

Find reviews written by "`John`" that mention "`excellent`".

Combining **Exact Matching** and **Full-Text Search**

- **Exact Filtering (`filter`)**: Select reviews where reviewer_name is exactly "John" using a term query.
- **Full-Text Search (`must`)**: Search for "excellent" in review_text using a match query.
- **Relevance Ranking:** Results are ranked according to their textual relevance, independently of the reviewer filter.

Key idea: Combine structured metadata constraints with full-text search in a single query.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "must": [
                {"match": {"review_text": "excellent"}}
            ],
            "filter": [
                {"term": {"reviewer_name": "John"}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q7: Combining Range Filtering and Full-Text Search

Find reviews with a rating higher than 3 that mention "`recommend`".

Combining **Numeric Constraints** and **Full-Text Search**

- **Range Filtering (filter)**: Select reviews where rating is greater than 3 using a range query with gt.
- **Full-Text Search (must):** Search for "recommend" in the review_text field using a match query.
- **Relevance Ranking**: Only the full-text query contributes to _score; the numeric constraint does not affect relevance scoring.

Key idea: Combine structured numeric constraints with relevance-based text retrieval.

Q7: Combining Range Filtering and Full-Text Search

Find reviews with a rating higher than 3 that mention "`recommend`".

Combining **Numeric Constraints** and **Full-Text Search**

- **Range Filtering (filter)**: Select reviews where rating is greater than 3 using a range query with gt.
- **Full-Text Search (must):** Search for "recommend" in the review_text field using a match query.
- **Relevance Ranking**: Only the full-text query contributes to _score; the numeric constraint does not affect relevance scoring.

Key idea: Combine structured numeric constraints with relevance-based text retrieval.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "must": [
                {"match": {"review_text": "recommend"}}
            ],
            "filter": [
                {"range": {"rating": {"gt": 3}}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q8: Prefix Search

Find reviews for movies whose titles start with the word "The".

Searching with a Prefix Query

- A **prefix query** retrieves documents whose indexed field value starts with a specified sequence of characters.
- Unlike match, prefix queries do not analyze the search text.
- The `movie_title` field is mapped as keyword, allowing prefix matching on the complete title.
- Prefix conditions can be used inside **bool.filter** when relevance ranking is not required.

Key idea: Prefix search matches the beginning of an indexed value, not its semantic meaning.

Q8: Prefix Search

Find reviews for movies whose titles start with the word "The".

Searching with a Prefix Query

- A **prefix query** retrieves documents whose indexed field value starts with a specified sequence of characters.
- Unlike match, prefix queries do not analyze the search text.
- The `movie_title` field is mapped as keyword, allowing prefix matching on the complete title.
- Prefix conditions can be used inside **bool.filter** when relevance ranking is not required.

Key idea: Prefix search matches the beginning of an indexed value, not its semantic meaning.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "filter": [
                {"prefix": {"movie_title": "The "}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q9: Boolean Search

Find reviews that mention both "`exciting`" and "`thrilling`", but do not mention "`boring`".

Combining Positive and Negative Conditions

- **must:** All specified conditions must match. These clauses contribute to relevance scoring.
- **must_not:** Excludes documents matching the specified conditions, without contributing to _score.
- **match:** Performs full-text searches on the analyzed review_text field.

Key idea: Boolean queries allow us to combine mandatory and exclusion conditions to refine search results.

Q9: Boolean Search

Find reviews that mention both "exciting" and "thrilling", but do not mention "boring".

Combining Positive and Negative Conditions

- **must:** All specified conditions must match. These clauses contribute to relevance scoring.
- **must_not:** Excludes documents matching the specified conditions, without contributing to _score.
- **match:** Performs full-text searches on the analyzed review_text field.

Key idea: Boolean queries allow us to combine mandatory and exclusion conditions to refine search results.

```
res = es.search(
    index="movie_reviews",
    size=3,
    query={
        "bool": {
            "must": [
                {"match": {"review_text": "exciting"}},
                {"match": {"review_text": "thrilling"}}
            ],
            "must_not": [
                {"match": {"review_text": "boring"}}
            ]
        }
    }
)

print(json.dumps(res.body, indent=4))
```

Q10: Aggregations

Calculate the average rating of all reviews for the movie "Avatar".

Analyzing Search Results with **Aggregations**

- **Exact Filtering (filter)**: Select reviews where movie_title is "Avatar".
- **Metric Aggregation (avg)**: Calculate the average value of the rating field across matching documents.
- **Aggregation Results:** Elasticsearch returns the computed statistic in the aggregations section of the response.
- **size=0**: Return only aggregation results, without retrieving individual document hits.

Key idea: Elasticsearch supports both information retrieval and analytical operations over matching documents.

Q10: Aggregations

Calculate the average rating of all reviews for the movie "Avatar".

Analyzing Search Results with **Aggregations**

- **Exact Filtering (filter)**: Select reviews where movie_title is "Avatar".
- **Metric Aggregation (avg)**: Calculate the average value of the rating field across matching documents.
- **Aggregation Results:** Elasticsearch returns the computed statistic in the aggregations section of the response.
- **size=0**: Return only aggregation results, without retrieving individual document hits.

Key idea: Elasticsearch supports both information retrieval and analytical operations over matching documents.

```
res = es.search(
    index="movie_reviews",
    size=0,
    query={
        "bool": {
            "filter": [
                {"term": {"movie_title": "Avatar"}}
            ]
        }
    },
    aggs={
        "average_rating": {
            "avg": {
                "field": "rating"
            }
        }
    }
)

print(json.dumps(res.body, indent=4))
print(res["aggregations"]["average_rating"]["value"])
```

Q11: Advanced Search and Aggregations

Q11: Which movies have the most highly rated reviews mentioning "amazing"?

Using the `movie_reviews` index, write an Elasticsearch query that:

1. Retrieves reviews containing the word "amazing" in review_text.
2. Considers only reviews with a rating greater than or equal to 4.
3. Groups the matching reviews by movie_title.
4. Calculates the number of matching reviews and their average rating for each movie.
5. Returns the top 5 movies, ordered by the number of matching reviews.

Bonus Challenge: Modify the query to rank movies by their average rating instead. Require at least 3 matching reviews per movie.

Hint: Combine `bool`, `match`, `range`, `terms`, and `avg` aggregations.

```
res = es.search(
    index="movie_reviews",
    size=0,
    query={
        "bool": {
            "must": [
                {"match": {"review_text": "amazing"}}
            ],
            "filter": [
                {"range": {"rating": {"gte": 4}}}
            ]
        }
    },
    aggs={
        "movies": {
            "terms": {
                "field": "movie_title",
                "size": 5,
                "min_doc_count": 1,
                "order": {"average_rating": "desc"}
            },
            "aggs": {
                "average_rating": {
                    "avg": {"field": "rating"}
                }
            }
        }
    }
)

for movie in res["aggregations"]["movies"]["buckets"]:
    print(
        f"{movie['key']}: "
        f"{movie['doc_count']} reviews, "
        f"average rating = {movie['average_rating']['value']:.2f}"
    )
```

Evaluating Retrieval Quality

A successful query does not necessarily mean that the retrieved documents are relevant.

How can we evaluate a retrieval system?

1. **Create a test set:** Define representative queries and identify their relevant documents (ground truth).
2. **Run the queries:** Retrieve the top-k results using the search engine.
3. **Measure retrieval quality**:
    1. **Precision@k**: Proportion of retrieved documents that are relevant.
    2. **Recall@k**: Proportion of all relevant documents retrieved in the top-k.
    3. **Latency**: Time required to execute the query.

Example:

A search retrieves 5 documents, of which 3 are relevant. There are 4 relevant documents in the entire collection.

- Precision@5 = 3/5 = 0.60
- Recall@5 = 3/4 = 0.75

Key idea: Retrieval quality depends on finding relevant information, not simply matching query terms or obtaining high relevance scores.

Hands-On Challenge

Index Your Own Dataset

Apply what you have learned to a new dataset containing Spanish texts.

1. **Choose a dataset**
   1. Find a publicly available dataset containing texts in Spanish (news, reviews, social media posts, etc.).
   2. Select at least 100 records, including a text field and some metadata.

2. **Prepare and index the data**
   1. Clean and validate the dataset.
   2. Define an Elasticsearch mapping with appropriate field types.
   3. Use the Spanish analyzer for full-text indexing.
   4. Load the documents into Elasticsearch.

3. **Explore and query the index**
   1. Perform a full-text search using match.
   2. Apply at least one metadata filter using term or range.
   3. Combine full-text search and filtering in a bool query.

4. **Inspect the results**
   1. Examine the retrieved documents and their relevance scores (_score).
   2. Discuss whether the results are relevant to the queries.

Expected outcome: A Jupyter notebook demonstrating data preparation, indexing, querying and retrieval results.

Bonus Challenge: Compare search results using the standard and spanish analyzers.

…What happens when relevant documents do not contain the same words as the user's query?



POLITÉCNICA

UNIVERSIDAD
POLITÉCNICA
DE MADRID



Universidad
Politécnica
de Madrid

ETSI **SISTEMAS INFORMÁTICOS**